JOB PORTAL MANAGEMENT SYSTEM

Python Mini Project Report — Complete Source Code Edition

Submitted by: _____

Roll Number: _____

Course / Section: _____

Subject: Python Programming

Submitted to: _____

Project technologies: Python 3, SQLite, command-line interface, hashlib, datetime

Project type: Academic mini project and case-study implementation

Department of Computer Applications

This report contains the project overview, case study, design, implementation details, test plan, output illustrations, and the complete 399-line Python source code.

1. Abstract

The Job Portal Management System is a menu-driven Python application designed to organize basic recruitment activities for job seekers and employers. Job seekers can create accounts, browse and search job listings, apply for suitable jobs by providing a resume file reference or portfolio link, write a short cover letter, and view application progress. Employers can publish vacancies, review the jobs they have posted, inspect applicants, and update application status. The application uses SQLite, a database engine included with Python, to store user, job, and application records between sessions. Passwords are hashed before storage, SQL queries use parameter placeholders, and role-specific menus separate job-seeker and employer activities. The system is intended as an educational prototype and can be extended into a web-based recruitment platform.

Project objectives

• Develop a working job portal using core Python concepts and a persistent database.

• Provide separate workflows for job seekers and employers.

• Store user accounts, job vacancies, and applications in relational tables.

• Support keyword-based job searching and application-status tracking.

• Demonstrate validation, exception handling, SQL joins, and password hashing.

2. Introduction and Case Study

Recruitment information is often exchanged through separate messages, spreadsheets, and email threads. Candidates may find it difficult to keep track of suitable vacancies and application outcomes. Employers may need to organize job descriptions, applicant details, resume references, and selection decisions in one place. A small centralized job portal can make these tasks more structured and easier to review.

2.1 Problem statement

The problem addressed by this project is the lack of a single, simple record system for publishing vacancies and tracking applications in a small recruitment workflow. Manual tracking can lead to duplicated records, delayed status updates, and difficulty locating job information.

2.2 Proposed solution

The proposed solution is a local Python program with role-based menus and an SQLite database. Employers can add job details and review applicants. Job seekers can search the job list, submit an application, and check its status. The database stores the information so that it remains available when the application is restarted.

3. Requirements Analysis

3.1 Functional requirements

• Account registration: a user supplies name, username, password, and account role.

• Login: the program verifies credentials and identifies the user's role.

• Guest browsing: visitors can view available jobs without signing in.

• Job listing: vacancies show title, company, location, salary range, job type, description, date, and employer.

• Job search: users can search by job title, company, location, or description keyword.

• Job posting: employers can publish vacancies after providing all required fields.

• Job application: job seekers submit a resume reference and cover letter for a selected job.

• Application history: job seekers can view the status and date of each application.

• Applicant review: employers can view applications for jobs they own.

• Status management: employers can update an application to Submitted, Under Review, Shortlisted, Rejected, or Hired.

3.2 Non-functional requirements

• Usability: the text menu should be readable and provide clear prompts.

• Reliability: SQLite transactions save records between sessions.

• Security: passwords are stored as salted hashes rather than plain text.

• Data integrity: unique usernames and unique job/applicant application pairs prevent duplicate records.

• Maintainability: functions separate registration, login, searching, posting, and application tasks.

• Portability: the project uses Python standard-library modules and does not require third-party packages.

3.3 Hardware and software requirements

Hardware: a standard laptop or desktop capable of running Python. Software: Python 3.8 or later is recommended; SQLite is accessed through the built-in sqlite3 module. A terminal or command prompt is required. The project can be edited in IDLE, Visual Studio Code, PyCharm, or another Python editor.

4. Project Design and Implementation

4.1 System workflow

Start program → initialize database and demo employer → display main menu → register, log in, browse as guest, or exit → validate user input → run the selected operation → show the result → return to a menu or close the program.

4.2 Main modules

• hash_password() and verify_password(): protect and verify passwords using PBKDF2-HMAC with SHA-256 and a random salt.

• connect_db() and initialize_database(): open SQLite connections, create tables, and add a demo employer account when needed.

• register() and login(): manage account creation and authentication.

• list_jobs() and search_jobs(): display vacancies and filter them using a keyword.

• post_job(): save employer-provided vacancy details.

• apply_for_job() and view_my_applications(): submit applications and display application history.

• view_employer_jobs(), view_applicants(), and update_application_status(): support employer-side management.

4.3 Database design

The database contains three related tables. The users table stores account details and role. The jobs table stores vacancies and identifies the employer who posted each job. The applications table connects a job with a job seeker and stores the resume reference, cover letter, status, and application date.

Table	Important fields	Purpose
users	id, full_name, username, password_hash, role	Account and role details; username is unique.

Table	Important fields	Purpose
jobs	id, employer_id, title, company, location, salary, job_type, description, posted_date	Job posting details and employer ownership.
applications	id, job_id, applicant_id, resume_link, cover_letter, status, applied_at	Application details; unique(job_id, applicant_id) prevents duplicate app

4.4 Data validation and security

The program checks that required registration and job-posting fields are not empty. It validates menu selections and converts a job ID to an integer with exception handling. SQL statements use placeholders to avoid building queries by concatenating user-entered values. Passwords are hashed with a random salt and PBKDF2-HMAC. The application table has a uniqueness constraint so the same job seeker cannot apply to the same job more than once.

4.5 Known limitations

This is a console-based local prototype. Resume data is stored as a file name or link rather than uploading the file. The project does not include email verification, password reset, automated recommendations, an administrator dashboard, or public web hosting. Production use would require stronger account recovery, access-control review, secure deployment, backup procedures, privacy notices, and monitoring.

5. How to Run the Project

• Install Python 3.8 or later.

• Save job_portal.py in a folder of your choice.

• Open Command Prompt or Terminal in that folder.

• Run the command: python job_portal.py

• The program creates job_portal.db automatically in the same folder.

• Use the demo employer account to test employer features: username employer; password employer123.

• Create a separate Job Seeker account to test job searching, applying, and application tracking.

Note: the demo credentials are for classroom testing only. Do not reuse this password for a real service.

6. Output / Screenshots

The images below are illustrative terminal-style examples using fictional sample data. They show the intended interaction flow; replace them with captures from your own execution if your instructor requires genuine runtime screenshots.

Figure 1. Main menu and available entry points.

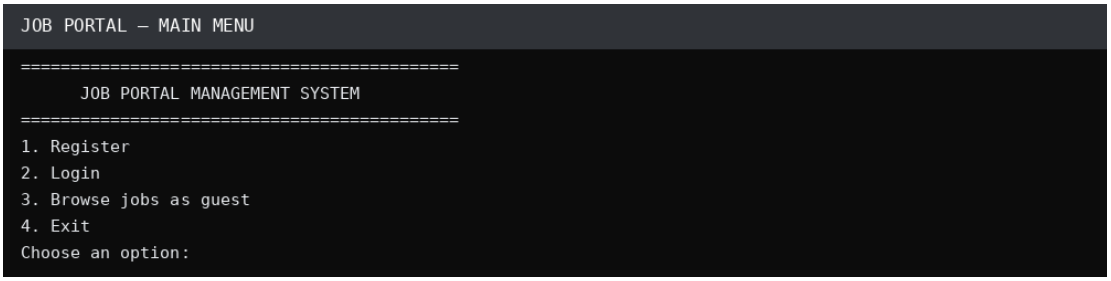


Figure 2. Employer enters vacancy details.

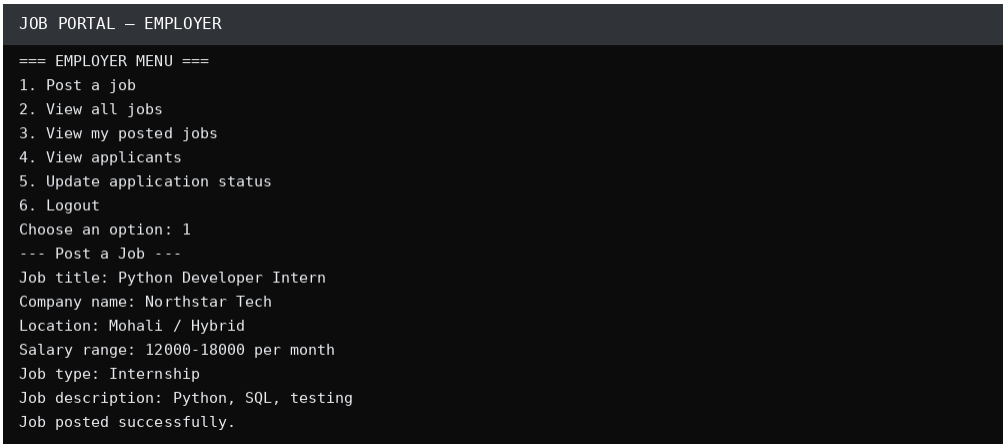


Figure 3. Available jobs displayed to users.

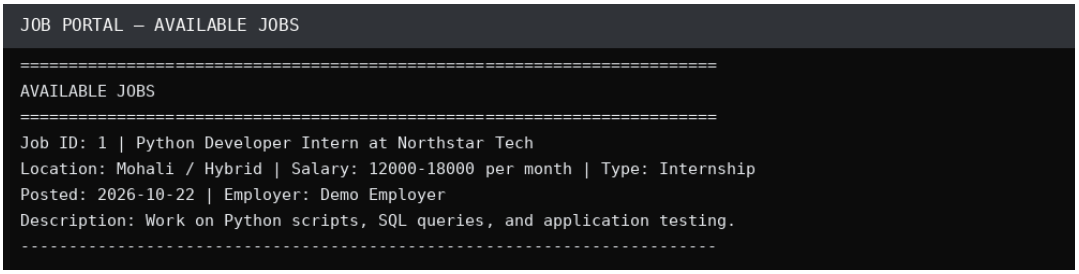
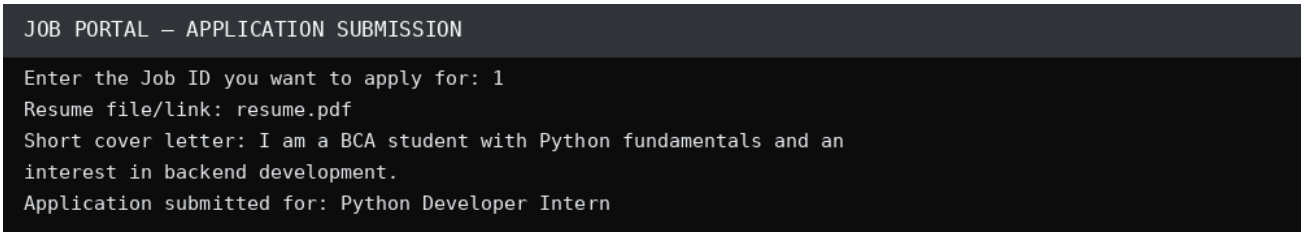


Figure 4. Job seeker submits an application.



6. Output / Screenshots (continued)

Figure 5. Job seeker checks application history.

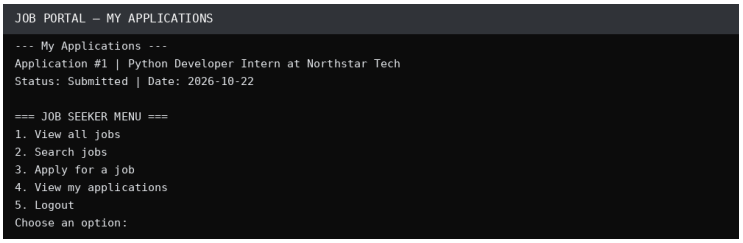


Figure 6. Employer reviews applicant information.

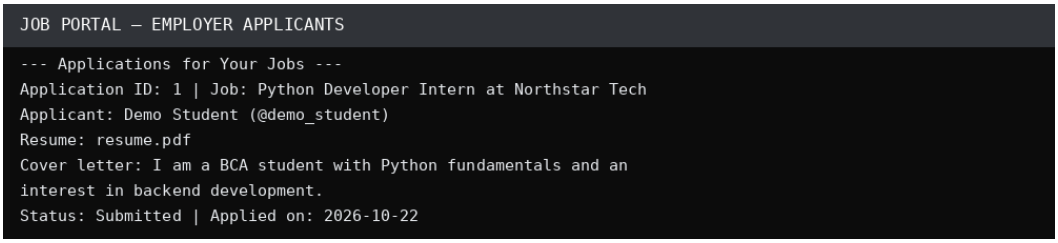


Figure 7. Employer updates application status.



7. Test Plan and Expected Results

Test	Action	Expected result
1	Register with valid details	Account is created.
2	Reuse a username	Duplicate username is rejected.
3	Post a complete vacancy	Job is stored and listed.
4	Search a keyword	Matching jobs are displayed.
5	Apply twice to the same job	Second application is rejected.
6	Update application status	New status is saved and displayed.

8. Conclusion and Future Scope

The Job Portal Management System demonstrates how Python and SQLite can support a structured recruitment workflow. It combines account registration, role-based access, job searching, job posting, applications, and status updates. The modular code and relational database make it suitable for an academic mini project. Future work can include a Flask or Django web interface, secure resume uploads, email notifications, stronger administrator controls, filters for experience and job category, and deployment with regular backups and privacy safeguards.

9. GitHub Repository Link

Repository URL: _____

After creating a GitHub repository, upload job_portal.py, README.md, and the screenshots folder. Paste the real repository URL above; no repository URL has been invented for this report.

10. Complete Python Source Code

The full source code used for this project is reproduced below. It is the complete program file supplied with the project package. Code is formatted in a compact monospaced style to preserve readability.

```
"""
JOB PORTAL MANAGEMENT SYSTEM
Python Mini Project | Uses SQLite (built into Python)

How to run:
1. Save this file as job_portal.py
2. Open a terminal in the file's folder.
3. Run: python job_portal.py

The database file (job_portal.db) is created automatically.
Demo employer account:
    username: employer
    password: employer123
"""

import sqlite3
import hashlib
import os
from datetime import datetime

DB_NAME = "job_portal.db"


def hash_password(password):
    """Hash a password before storing it in the database."""
    salt = os.urandom(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, 120_000)
    return salt.hex() + ":" + digest.hex()


def verify_password(password, stored_value):
    try:
        salt_hex, digest_hex = stored_value.split(":")
        salt = bytes.fromhex(salt_hex)
        digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, 120_000)
        return digest.hex() == digest_hex
    except (ValueError, TypeError):
        return False


def connect_db():
    return sqlite3.connect(DB_NAME)


def initialize_database():
    with connect_db() as conn:

        cur = conn.cursor()
        cur.execute("""
            CREATE TABLE IF NOT EXISTS users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                full_name TEXT NOT NULL,
                username TEXT UNIQUE NOT NULL,
                password_hash TEXT NOT NULL,
                role TEXT NOT NULL CHECK(role IN ('jobseeker', 'employer'))
            )
        """)
        cur.execute("""
            CREATE TABLE IF NOT EXISTS jobs (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                employer_id INTEGER NOT NULL,
                title TEXT NOT NULL,
                company TEXT NOT NULL,
                location TEXT NOT NULL,
                salary TEXT NOT NULL,
                job_type TEXT NOT NULL,
                description TEXT NOT NULL,
                posted_on TEXT NOT NULL,
                FOREIGN KEY(employer_id) REFERENCES users(id)
            )
        """)
        cur.execute("""
            CREATE TABLE IF NOT EXISTS applications (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                job_id INTEGER NOT NULL,
                applicant_id INTEGER NOT NULL,
                resume_link TEXT NOT NULL,
                cover_letter TEXT NOT NULL,
                status TEXT NOT NULL DEFAULT 'Submitted',
                applied_on TEXT NOT NULL,
                UNIQUE(job_id, applicant_id),
                FOREIGN KEY(job_id) REFERENCES jobs(id),
                FOREIGN KEY(applicant_id) REFERENCES users(id)
            )
        """)
        # Add a demo employer only when the database is first initialized.
        cur.execute("SELECT id FROM users WHERE username = ?", ("employer",))
        if cur.fetchone() is None:
            cur.execute(
                "INSERT INTO users (full_name, username, password_hash, role) VALUES (?, ?, ?, ?)",
                ("Demo Employer", "employer", hash_password("employer123"), "employer")
            )

def register():
    print("\n--- Create Account ---")
    name = input("Full name: ").strip()
    username = input("Choose username: ").strip()
    password = input("Choose password: ")
    role = input("Register as (1) Job Seeker or (2) Employer: ").strip()
    if not name or not username or not password:
```

```
        print("All fields are required.")
        return
    if role not in ("1", "2"):
        print("Please select 1 or 2.")
        return

    role_name = "jobseeker" if role == "1" else "employer"
    try:
        with connect_db() as conn:
            conn.execute(
                "INSERT INTO users (full_name, username, password_hash, role) VALUES (?, ?, ?, ?)",
                (name, username, hash_password(password), role_name)
            )
        print("Account created successfully. You can now log in.")
    except sqlite3.IntegrityError:
        print("That username is already taken. Please choose another.")


def login():
    print("\n--- Login ---")
    username = input("Username: ").strip()
    password = input("Password: ")
    with connect_db() as conn:
        row = conn.execute(
            "SELECT id, full_name, username, password_hash, role FROM users WHERE username = ?",
            (username,)
        ).fetchone()

    if row and verify_password(password, row[3]):
        print(f"\nWelcome, {row[1]}!")
        return {"id": row[0], "name": row[1], "username": row[2], "role": row[4]}
    print("Invalid username or password.")
    return None


def list_jobs(search_term=None):
    with connect_db() as conn:

        if search_term:
            pattern = f"%{search_term}%"
            rows = conn.execute("""
                SELECT j.id, j.title, j.company, j.location, j.salary, j.job_type,
                       j.description, j.posted_on, u.full_name
                FROM jobs j JOIN users u ON j.employer_id = u.id
                WHERE j.title LIKE ? OR j.company LIKE ? OR j.location LIKE ?
                      OR j.description LIKE ?
                ORDER BY j.id DESC
            """, (pattern, pattern, pattern, pattern)).fetchall()
        else:
            rows = conn.execute("""
                SELECT j.id, j.title, j.company, j.location, j.salary, j.job_type,
                       j.description, j.posted_on, u.full_name
                FROM jobs j JOIN users u ON j.employer_id = u.id
                ORDER BY j.id DESC
            """).fetchall()

    if not rows:
        print("No jobs found.")
        return []

    print("\n" + "=" * 72)
    print("AVAILABLE JOBS")
    print("=" * 72)
    for job in rows:
        print(f"\nJob ID: {job[0]} | {job[1]} at {job[2]}")
        print(f"Location: {job[3]} | Salary: {job[4]} | Type: {job[5]}")
        print(f"Posted: {job[7]} | Employer: {job[8]}")
        print(f"Description: {job[6]}")
        print("-" * 72)
    return rows


def search_jobs():
    term = input("Enter job title, company, location, or keyword: ").strip()
    if term:
        list_jobs(term)
    else:
        print("Please enter a search term.")


def post_job(user):
    print("\n--- Post a Job ---")
    title = input("Job title: ").strip()
    company = input("Company name: ").strip()

    location = input("Location (or Remote): ").strip()
    salary = input("Salary range: ").strip()
    job_type = input("Job type (Full-time/Part-time/Internship/Remote): ").strip()
    description = input("Job description and requirements: ").strip()

    if not all([title, company, location, salary, job_type, description]):
        print("All fields are required.")
        return

    with connect_db() as conn:
        conn.execute("""
            INSERT INTO jobs
            (employer_id, title, company, location, salary, job_type, description, posted_on)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """, (user["id"], title, company, location, salary, job_type, description,
              datetime.now().strftime("%Y-%m-%d")))
    print("Job posted successfully.")


def apply_for_job(user):
    list_jobs()
    try:
        job_id = int(input("\nEnter the Job ID you want to apply for: ").strip())
```

```
    except ValueError:
        print("Job ID must be a number.")
        return

    with connect_db() as conn:
        job = conn.execute("SELECT id, title FROM jobs WHERE id = ?", (job_id,)).fetchone()
    if not job:
        print("That job does not exist.")
        return

    resume_link = input("Resume file/link (e.g. resume.pdf or portfolio URL): ").strip()
    cover_letter = input("Short cover letter: ").strip()
    if not resume_link or not cover_letter:
        print("Resume reference and cover letter are required.")
        return

    try:
        with connect_db() as conn:
            conn.execute("""
                INSERT INTO applications
                (job_id, applicant_id, resume_link, cover_letter, status, applied_on)
                VALUES (?, ?, ?, ?, 'Submitted', ?)
            """, (job_id, user["id"], resume_link, cover_letter,

                  datetime.now().strftime("%Y-%m-%d")))
        print(f"Application submitted for: {job[1]}")
    except sqlite3.IntegrityError:
        print("You have already applied for this job.")


def view_my_applications(user):
    with connect_db() as conn:
        rows = conn.execute("""
            SELECT a.id, j.title, j.company, a.status, a.applied_on
            FROM applications a JOIN jobs j ON a.job_id = j.id
            WHERE a.applicant_id = ? ORDER BY a.id DESC
        """, (user["id"],)).fetchall()
    print("\n--- My Applications ---")
    if not rows:
        print("You have not applied for any jobs yet.")
        return
    for row in rows:
        print(f"Application #{row[0]} | {row[1]} at {row[2]} | Status: {row[3]} | Date: {row[4]}")


def view_employer_jobs(user):
    with connect_db() as conn:
        rows = conn.execute("""
            SELECT id, title, company, location, posted_on
            FROM jobs WHERE employer_id = ? ORDER BY id DESC
        """, (user["id"],)).fetchall()
    print("\n--- Jobs Posted by You ---")
    if not rows:
        print("You have not posted any jobs.")
        return
    for row in rows:
        print(f"Job ID: {row[0]} | {row[1]} at {row[2]} | {row[3]} | Posted: {row[4]}")


def view_applicants(user):
    with connect_db() as conn:
        rows = conn.execute("""
            SELECT a.id, j.title, j.company, u.full_name, u.username,
                   a.resume_link, a.cover_letter, a.status, a.applied_on
            FROM applications a
            JOIN jobs j ON a.job_id = j.id
            JOIN users u ON a.applicant_id = u.id
            WHERE j.employer_id = ?
            ORDER BY a.id DESC
        """, (user["id"],)).fetchall()

    print("\n--- Applications for Your Jobs ---")
    if not rows:
        print("No applications received yet.")
        return
    for row in rows:
        print(f"\nApplication ID: {row[0]} | Job: {row[1]} at {row[2]}")
        print(f"Applicant: {row[3]} (@{row[4]})")
        print(f"Resume: {row[5]}")
        print(f"Cover letter: {row[6]}")
        print(f"Status: {row[7]} | Applied on: {row[8]}")
        print("-" * 60)


def update_application_status(user):
    view_applicants(user)
    try:
        app_id = int(input("\nEnter Application ID to update: ").strip())
    except ValueError:
        print("Application ID must be a number.")
        return

    print("1. Submitted  2. Under Review  3. Shortlisted  4. Rejected  5. Hired")
    choices = {
        "1": "Submitted", "2": "Under Review", "3": "Shortlisted",
        "4": "Rejected", "5": "Hired"
    }
    choice = input("Choose new status: ").strip()
    if choice not in choices:
        print("Invalid status choice.")
        return

    with connect_db() as conn:
        cur = conn.execute("""
            UPDATE applications SET status = ?
            WHERE id = ? AND job_id IN (SELECT id FROM jobs WHERE employer_id = ?)
        """, (choices[choice], app_id, user["id"]))
    if cur.rowcount:
```

```
        print("Application status updated.")
    else:
        print("Application not found under your posted jobs.")


def jobseeker_menu(user):
    while True:
        print("\n=== JOB SEEKER MENU ===")
        print("1. View all jobs")
        print("2. Search jobs")
        print("3. Apply for a job")
        print("4. View my applications")
        print("5. Logout")
        choice = input("Choose an option: ").strip()
        if choice == "1":
            list_jobs()
        elif choice == "2":
            search_jobs()
        elif choice == "3":
            apply_for_job(user)
        elif choice == "4":
            view_my_applications(user)
        elif choice == "5":
            break
        else:
            print("Invalid option. Try again.")


def employer_menu(user):
    while True:
        print("\n=== EMPLOYER MENU ===")
        print("1. Post a job")
        print("2. View all jobs")
        print("3. View my posted jobs")
        print("4. View applicants")
        print("5. Update application status")
        print("6. Logout")
        choice = input("Choose an option: ").strip()
        if choice == "1":
            post_job(user)
        elif choice == "2":
            list_jobs()
        elif choice == "3":
            view_employer_jobs(user)
        elif choice == "4":
            view_applicants(user)
        elif choice == "5":
            update_application_status(user)
        elif choice == "6":
            break
        else:
            print("Invalid option. Try again.")


def main():
    initialize_database()
    while True:
        print("\n" + "=" * 44)
        print("      JOB PORTAL MANAGEMENT SYSTEM")
        print("=" * 44)
        print("1. Register")
        print("2. Login")
        print("3. Browse jobs as guest")
        print("4. Exit")
        choice = input("Choose an option: ").strip()

        if choice == "1":
            register()
        elif choice == "2":
            user = login()
            if user:
                if user["role"] == "jobseeker":
                    jobseeker_menu(user)
                else:
                    employer_menu(user)
        elif choice == "3":
            list_jobs()
        elif choice == "4":
            print("Thank you for using the Job Portal Management System.")
            break
        else:
            print("Invalid option. Please choose 1-4.")


if __name__ == "__main__":
    main()
```